

РЕФЕРАТ

Выпускная квалификационная работа содержит: 42 страницы, 11 рисунков, список использованных источников содержит 7 позиций, приводится 1 приложение.

АРХИТЕКТУРА ВЕБ-ПРИЛОЖЕНИЯ, SCRAPY, КЛАССИФИКАЦИЯ, SQ LITE, REACT, DJANGO.

Работа посвящена разработке программного обеспечения для интеллектуального анализа вакансий и построения плана для подготовки к собеседованию (на примере вакансий в IT-отрасли).

В теоретической части исследуются выбранные инструменты для разработки. В результате, на основе проведённого анализа сделан выбор набора технологий, оптимально подходящих для разработки данного сервиса (Для реализации интерфейса был выбран язык программирования JavaScript с использованием библиотек React и Bootstrap, для программно-аппаратной части – фреймворк Django, Rest API и СУБД SQL Lite, для сбора данных решено использовать Scrapy)

В практической части представленной работы описан процесс сбора, обработки, анализа и хранения вакансий IT-специалистов. Также представлены этапы разработки приложения с приложенными схемами и диаграммами, а также пользовательский интерфейс для пользователей сервиса.

СОДЕРЖАНИЕ

РЕФЕРАТ.....	2
ВВЕДЕНИЕ	4
ОСНОВНАЯ ЧАСТЬ	6
1 ТЕОРЕТИЧЕСКАЯ ЧАСТЬ.....	7
1.1 Постановка задачи	7
1.2 Frontend.....	8
1.2.1 Язык разметки HTML.....	8
1.2.2 Таблицы стилей CSS	9
1.2.3 Язык программирования JavaScript	10
1.5 Backend	12
1.5.1 Программный интерфейс приложения	13
1.5.2 Схема MVC	14
1.5.3 Технология ORM	16
1.5.4 Web-сервер	17
1.6 Базы данных	19
1.6.1 Основные понятия	19
1.6.2 Модели данных	22
2 ПРАКТИЧЕСКАЯ ЧАСТЬ.....	22
2.1 Проектирование архитектуры	22
2.1.1 Архитектура приложения	23
2.1.2 Схема базы данных.....	24
2.2 Разработка практических решений	26
2.2.1 Этап сбора данных.....	26
2.2.2 Создание классификатора.....	28
2.2.3 Frontend.....	29
2.2.4 Backend	33
2.2.5 Результат работы	36
ЗАКЛЮЧЕНИЕ.....	38
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	39
ПРИЛОЖЕНИЯ	40
ПРИЛОЖЕНИЕ 1	41

ВВЕДЕНИЕ

Выпускная квалификационная работа посвящена разработке веб-сервиса интеллектуального анализа вакансий и построения плана для подготовки к собеседованию (на примере вакансий в IT-отрасли). Объектом исследования является организация сбора, обработки, анализа и хранения данных – вакансий IT-специалистов.

В условиях нынешних реалий нельзя представить свою жизнь без использования глобальной сети Интернет, она является неотъемлемой частью жизни миллиардов людей. Темпы ее развития не останавливаются никогда, всё больше сфер жизни переводятся в дистанционный формат. Интернет проник в жизнь каждого человека, такой огромный спрос является движущей силой в развитии новых технологий, а исполнителями являются огромное количество крупных IT-компаний. Данная сфера на протяжении последнего десятилетия является самой быстроразвивающейся и одной из самых прибыльных, так как информационные потребности населения постоянно растут.

Программное обеспечение делает жизнь людей комфортнее. Оно используется в огромном количестве сфер жизни, таких как образование, промышленность, медицина, не говоря уже о сферах, которые изначально были развернуты в информационной сфере. Стоит отметить, что используемые продукты не всегда отличаются быстротой работы и функциональностью, поэтому всегда нуждаются в доработке и поиске новых решений.

Технологии в наше время развиваются с огромной скоростью, поэтому разработчики всегда должны быстро совершенствовать свой продукт, ориентируясь на запросы пользователей и ситуацию на рынке. Именно поэтому ПО для разработки также является важной частью в совокупности ПО в целом. Появляются новые обновления и библиотеки, облегчающую жизнь разработчикам и оптимизирующие сам процесс разработки.

Задача поиска работы значительно облегчилась с появлением направленных на это платформ. С помощью сайтов, таких как HeadHunter,

теперь можно сэкономить большое количество времени. Есть возможность откликнуться на вакансию, приложить резюме и дожидаться ответа работодателя. Однако, что делать, когда работодатель рассмотрел вашу кандидатуру и пригласил на собеседование. Как подготовиться к нему? Что говорить?

Целью данной работы как раз и является разработка web-сервиса для подготовки к собеседованию. Такой сервис должен помочь людям успешно пройти собеседование и попасть на работу по конкретной вакансии с сайта поиска работы.

ОСНОВНАЯ ЧАСТЬ

1 ТЕОРЕТИЧЕСКАЯ ЧАСТЬ

1.1 Постановка задачи

В рамках данной выпускной квалификационной работы предполагается разработать сервис (веб-приложение) интеллектуального анализа вакансий и построения плана для подготовки к собеседованию (на примере вакансий в IT-отрасли

Веб-приложение имеет определенную структуру. Оно состоит из частей, каждая из которых несет в себе исчерпывающий функционал и отвечает за этапы работы приложения от взаимодействия с пользователем до реализации алгоритмов и хранения данных.

1. Frontend – клиентская сторона пользовательского интерфейса. Отвечает за взаимодействие пользователя с программно-аппаратной частью приложения. В моём случае это интерфейс, выполненный в стиле тематического сайта, на котором помимо основного функционала есть несколько вкладок, которые связаны с темой поиска работы и сопутствующих ему этапов. Для реализации этой идеи был выбран язык программирования JavaScript с использованием библиотек React и Bootstrap.
2. Backend – программно-аппаратная часть приложения. Эта часть отвечает за хранение и анализ данных. В данном случае используется фреймворк Django, с реализацией основной логики на языке программирования Python.
3. API (Application Program Interface) – программный интерфейс приложения. Он реализует связь графического интерфейса приложения (Frontend) и его программно-аппаратной части (Backend), то есть служит способом их взаимодействия между собой.
4. База данных. Этот фрагмент архитектуры реализует функцию хранения и формирования выборки данных для разработанного приложения. Данная часть должна быстро и качественно обрабатывать

и загружать данные, а также обладать функционалом для работы с ними. Было решено использовать встроенную систему управления базами данных SQ Lite.

1.2 Frontend

Frontend – графический интерфейс приложения. Он является неотъемлемой частью архитектуры веб-приложения, ведь без него сложно представить как пользователь сможет взаимодействовать с программно-аппаратной частью. Именно наличие в этой цепочке пользователя накладывает ряд требований к frontend'у. Интерфейс должен быть интуитивно-понятным, простым и функциональным, иначе обыкновенный человек не будет им пользоваться.

Не смотря на простое описание, графический интерфейс – это достаточно сложный механизм, в котором применяется большое количество технологий, которые требуют от разработчика обширных знаний и опыта. Сфера веб-разработки слишком быстро меняется, это является причиной сложности поддержания и реализации больших проектов. Основными инструментами при Web-разработке являются: HTML, CSS, JavaScript с огромным количеством библиотек, фреймворков и шаблонов проектирования (React, JQuery, AJAX, SPA, Webpack). На некоторых из них можно остановиться и рассмотреть более подробно.

1.2.1 Язык разметки HTML

HTML (HyperText Markup Language)– язык гипертекстовой разметки для описания разметки различных документов. После того, как HTML-файл загружается в браузер, он преобразует его в формат, визуально просматриваемый на устройствах.

Стоит отметить, что существует более строгий формат данного языка разметки – XHTML. В нем решена проблема перекрытия тэгов (сущностей страницы).

В HTML документ описывается с помощью сущностей страницы – тегов. Документ состоит из объектов, которые находятся внутри символов тега (<...>). В этих элементах прописывается основная информация о стиле и оформлении страницы, именно они определяют каким внешним видом будет обладать документ при отображении его на экранах устройств. Теги могут быть и пустыми, но впоследствии заполняемыми данными.

Сейчас существует проблема того, что существует большое количество модификаций тегов HTML, что приводит к различиям в отображениях HTML-шаблонов в разных браузерах. Это осложняет работу разработчиков, ведь приложение должно поддерживать как можно большее их количество. Именно поэтому было решено сформировать определенный стандарт, следование которому при разработке позволяет облегчить создание кроссплатформенных веб-приложений.

1.2.2 Таблицы стилей CSS

CSS (Cascading Style Sheets) – каскадные таблицы стилей, язык описания графической визуальной составляющей HTML-документа.

Каскадные таблицы стилей – это инструмент для определения внешнего вида элементов страницы, например, задания стилей шрифтов, цветов фона, расположения объектов. CSS может показывать один документ через разные способы подачи информации: на цифровом экране устройств, в печатном формате, голосовым способом и т.д.

CSS в основном всегда используется только в связке HTML. Это две составляющие графического интерфейса, одна из них – логическая структура страницы, а вторая – визуальная. Благодаря такому подходу разделения

облегчается работа над приложением, увеличивается гибкость в разработке этих отдельных частей и оптимизируется количество написанного кода.

Стили можно связывать с шаблоном страницы как непосредственным добавлением кода в документ, так и хранением отдельного CSS файла, подключаемого через путь к основному HTML-шаблону

Использование формата CSS основано на двух принципах – каскадирование и наследование.

Наследование состоит в том, что элементы-потомки наследуют свойства и атрибуты от элементов-предков.

Каскадирование состоит в возможности соответствия нескольких стилей одному элементу. При этом они конфликтуют между собой, и этот конфликт решается с помощью правил приоритета. Самым высоким приоритетов обладают правила с пометкой `important`. Самым низким - правила стандартных стилей используемого браузера.

1.2.3 Язык программирования JavaScript

JavaScript – язык программирования, который поддерживает несколько парадигм программирования. Используя этот язык, у разработчика есть возможность выбрать между функциональным, императивным и объектно-ориентированными стилями. Данный язык применяется чаще всего именно в веб-разработке графического интерфейса. С помощью него можно «оживить» страницы или осуществить связку шаблона со статическими файлами.

JS по сути свой является больше объектно-ориентированным языком программирования, однако в нем отсутствует понятие класса. Как же тогда реализуется наследование? В этом ЯП оно происходит посредством копирования элемента-предка.

Однако, JS обладает и свойствами, типичными для функционального подхода: функции являются объектами, объекты представлены списками, функции с множеством аргументов представляются как множества функций с

одним аргументом, а также есть замыкания и анонимные функции. Из этого можно судить о гибкости JS, которая так ценится в разработке.

JS-код достаточно просто встраивается в HTML-шаблоны. Это делается с помощью вставки кода внутри тега «script», либо указания ссылки на файл с кодом в том же документе.

Синтаксис JS имеет сильные сходства с языком Си, но все-таки есть несколько различий, таких как возможность получения типа и структуры объекта во время выполнения программы, автоматическое очищение неиспользуемой программой памяти, анонимность функций, автоматизация приведения типов и возможность представления функций объектами.

С началом внедрения в современную разработку технологий AJAX, которые базируются на построении графических пользовательских интерфейсов с помощью «фонового» асинхронного обмена данными приложения с сервером, начал набирать популярность и язык Java Script. Такой подход позволяет не загружать снова и снова страницу браузера, что ускоряет и упрощает использование этой страницы.

Язык используется и при создании скриптов, с помощью которых можно выполнять на странице автоматическое заполнение форм для пользователей, форматировать страницу по своему желанию, определять отображаемое содержимое и управлять страницей и ее элементами в целом. Все эти скрипты выполняются сразу при загрузке шаблона страницы.

В языке JS существует большое количество встроенных библиотек, что также говорит о его функциональности и причине популярности у разработчиков. Эти библиотеки позволяют упростить большинство действий, и даже автоматизировать их. Как пример, библиотека JQuery обладает функционалом для создания AJAX запросов.

Сейчас практически все популярные веб-приложения являются одностраничными (SPA – Single Page Application). Разработка таких приложений основана на использовании одного HTML-шаблона, который является оболочкой для остальных страниц. HTML, JS и CSS в этом случае

являются динамически подгружаемыми объектами, а подгружаются они как раз с помощью технологии AJAX.

Основными сущностями в таких Web-приложениях являются: JS Фреймворк, который реализует приложение; HTML-документы, которые являются основой для динамических страниц; REST-API, осуществляющий взаимодействие с внутренней логикой серверной части и, непосредственно, сама технология AJAX

Среди разработчиков самыми часто используемыми фреймворками для SPA являются React.js, Angular и Vue.js.

1.5 Backend

Backend – это программно-аппаратная часть веб-приложения. Она также является неотъемлемой частью любого веб-сервиса. Именно она отвечает за внутреннюю логику, основой за которую берутся запросы и желания пользователя. Backend также как и визуальный интерфейс должен обладать несколькими неотъемлемыми характеристиками, такими как простота, грамотная оптимизация кода и безопасной базой данных. Если эти требования не будут выполнены – пользователям будет некомфортно пользоваться сервисом ввиду долгого ожидания ответов от сервера или дырами в конфиденциальности их данных.

Программно-аппаратная часть приложения – это достаточно сложный, объемный механизм, который требует от разработчика внимательности и компетентности в своих решениях. Необходимо грамотно выстроить связи, между составляющими сервиса. Также необходимо продумать движение данных, ведь успех приложения зависит от скорости и надежности их передачи и обработки. Для этих целей управления потоками существует большое количество фреймворков, на основе которых моделируются сценарии поведения.

Сфера веб-разработки стремительно развивается, появляются новые фреймворки и шаблоны сценариев. Вместе с тем происходит и укоренение проверенных методов разработки. Рассмотрим основной список средств и шаблонов для этих задач:

- API (Application Program Interface) – программный интерфейс приложения, совокупность методов коммуникации пользовательского интерфейса и внутренней логики приложения;
- В основном Backend программируется на данных ЯП: Python, C#, Java, Go, JavaScript, PHP;
- Необходимы инструменты для тестирования: PyUnit, TestNG, XUnit;
- Инструменты для связи с БД;
- Инструменты администрирования БД;
- Инструменты для работы с данными: Django;

Некоторые из них рассмотрим подробно.

1.5.1 Программный интерфейс приложения

Как говорилось ранее, API – это совокупность методов коммуникации пользовательского интерфейса и внутренней логики приложения.

Для облегчения описания предстоящей работы с сервисом, можно рассмотреть схему представления архитектуры MVC-приложения, которая показана на рис. 1.1

API отвечает за получение запросов от пользовательского интерфейса, а также за отправку ей ответа от программного интерфейса. Данный инструмент обычно представляется в форме определенного количества парных URL-запросов и, отвечающих за их обработку, контроллеров. Запрос анализируется, после чего система ищет подходящий ему контроллер, в последствие принимающий данные. Затем контроллером реализуются определенные кодом действия с данными, зачастую, с привязкой к базе

данных. Уже после этого конечный результат манипуляций возвращается обратно пользовательской части.

API можно назвать ещё одним уровнем абстракции над протоколами передачи данных во сети интернет (HTTP/HTTPS). Разработчики приложения с двух сторон заранее договариваются о стандартах обменных методов информации между частями приложения.

1.5.2 Схема MVC

Веб-приложение обязано выполнять следующие функции:

- Маршрутизация запроса. По получаемому URL-запросу приложение должно вызывать определенную функцию для его обработки;
- Оценка заголовков и параметров URL-запроса;
- Выполнение бизнес-логики. Приложение должно производить заданные бизнес-логикой действия, будь то какие-либо вычисления или доступ к данным в БД;
- Связь с базами данных;
- Отображение ответа, например в формате HTML-страницы;

Для такого достаточно большого объема задач необходимо использование Фреймворков, ведь без них их решение является сложным и трудоемким процессом.

MVC (Model-View-Controller) – схема модель-представление-контроллер, шаблон архитектуры, задающий структуру веб-приложения. Эта структура представлена на рис. 1.1. На ней показано из чего должен состоять веб-сервис, а также взаимодействие между его составляющими.

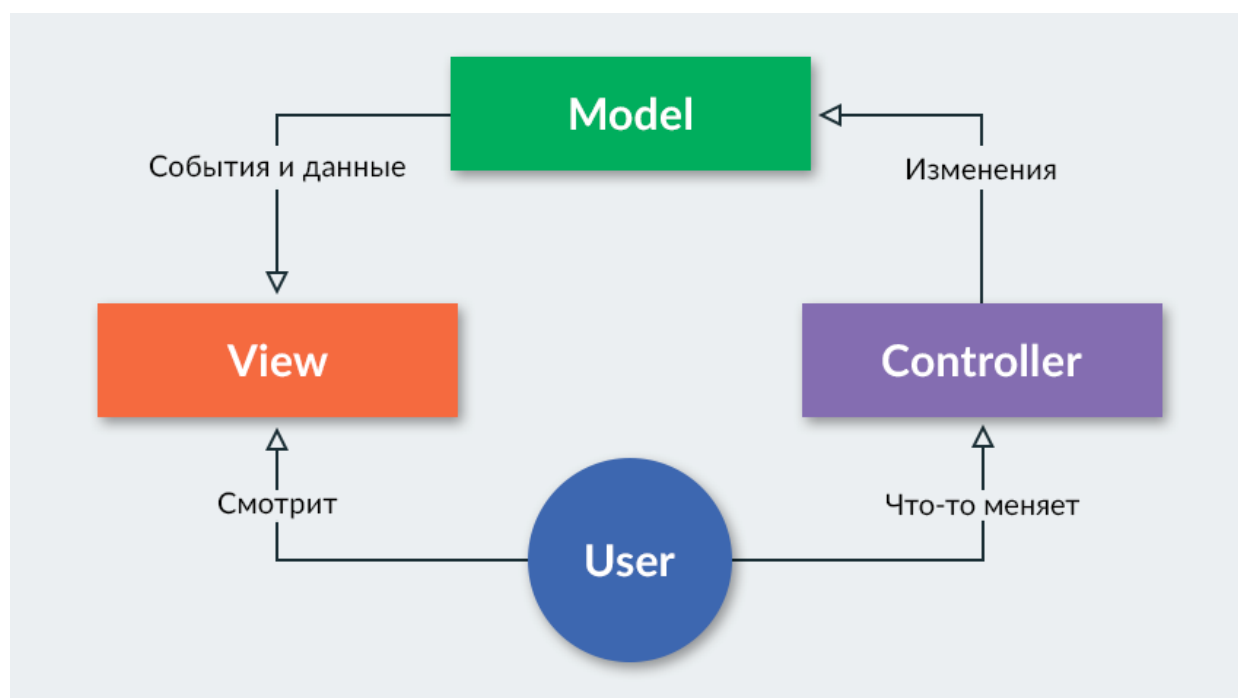


Рис. 1.1 - Архитектура MVC приложения

Необходимо выделить функцию, которая будет вызываться для каждого запроса, то есть определить начальную точку для входа в сервис. Для этой цели применяется маршрутизатор. Он отображает получаемый URL-запрос на соответствующую ему функцию обработки. То есть роутер по сути является описанием того, какая функция соответствует какому запросу.

Затем в дело вступает контроллер. В отличие от маршрутизатора, контроллеров в приложении всегда несколько. Они осуществляют обращение к модели данных, которая выполняет действия, основанные на бизнес-логике приложения. К таким действиям относятся: взаимодействие с базой данных и с внешними сервисами, программные вычисления и другие. После взаимодействия с моделью, она отдает контроллеру обработанные и визуализированные данные. Контроллер же, получив ответ, отдает их в View, где эти данные складываются в HTML-шаблон, который далее визуализируется. В итоге полученные данные на HTML-странице передаются обратно пользователю.

Сейчас существует большое разнообразие различных MVC Фреймворков: Django, Yii, Ruby on rails и другие. Данные Фреймворки обладают целым рядом преимуществ:

- Реализованная архитектура, которая делает работу программиста намного легче, избавляя от ошибок;
- Оптимизация использования кода. Существует большое количество готовых библиотек и шаблонов для решения готовых задач и экономии ресурсов;
- Доступная исчерпывающая документация;

1.5.3 Технология ORM

ORM (Object-Relational Mapping) – объектно-реляционное отображение, это инструмент, для связи сущности в БД с объектами в языках, придерживающихся концепции ООП.

Эта технология создавалась для облегчения разработки, так как с ней разработчик мог не писать SQL-запросы для работы с базой данных. Инструмент ORM также допускает возможность использования виртуальной объектной БД в серверной части приложения, он может форматировать информацию в базе данных, брать ее из базы, проводить различные действия с содержимым БД. Данный подход позволяет значительно уменьшить количество предполагаемых ошибок в разработке и оптимизировать написание запросов.

Некоторые из ORM могут кэшировать информацию, отдаваемую после запросов, что позволяет программной части приложения не обращаться каждый раз к базе данных при возникновении похожих запросов, а просто брать нужные данные из кэша. Этот подход значительно уменьшает время ответа сервиса.

Также на быстродействие сервиса положительно оказывает влияние и возможность автоматически синхронизировать информацию о сущностях и объектах в памяти с теми же объектами в базе данных.

У данного подхода есть и недостатки. Приведем некоторые из них: заведомо большие затраты памяти, низкая скорость работы в некоторых условиях. Однако большая часть ORM-технологий сейчас позволяет программисту возможность самостоятельно утвердить код SQL-запросов.

1.5.4 Web-сервер

Web-сервер – это сетевой сервер, реализующий обработку сетевого протокола HTTP.

При начале работы он считывает пакет конфигурации, где содержится информация о данных его работы. Если передаваемый файл неверен, то сервер не начнет работу, иначе – запускается. Веб-сервер записывает данные о процессе его работы, называемые логами. Формат логов – текстовый, пример лога – данные о возникающих в системе ошибках.

Сервер можно разделить на два процесса, которые работают в операционной системе. Мастер-процесс – это процесс, начинающий работу при запуске. Он отвечает за считывание конфигурационных пакетов, запуск сокетов и начало работы воркер-процессов. Когда последние начинают свою работу, мастер-процесс выполняет их поддержание на определенном количественном уровне и мониторинг информации о них. Функционал таких процессов заключается и в обработке получаемых запросов.

Принцип работы воркер-процесса по шагам:

1. Считывание полученного HTTP запроса;
2. Определение части конфигурационного пакета, занимающейся обработкой необходимого домена;

3. Выбор пути, более маленькой составляющей конфигурационного пакета, занимающейся обработкой определенной совокупности URL-запросов;
4. Доступ к пакету из операционной системы;
5. Считывание пакета;
6. Установка необходимых HTTP-заголовков;
7. Возвращение ответа;

Веб-сервер может возвращать и статические файлы. Как пример, юзер посещает сервис впервые, то есть ему не нужно использовать программно-аппаратную часть приложения. Логичнее и проще возвращать пользователю статический файл с HTTP-шаблоном страницы.

Этот формат запросов единственный, который обслуживается веб-сервером самостоятельно.

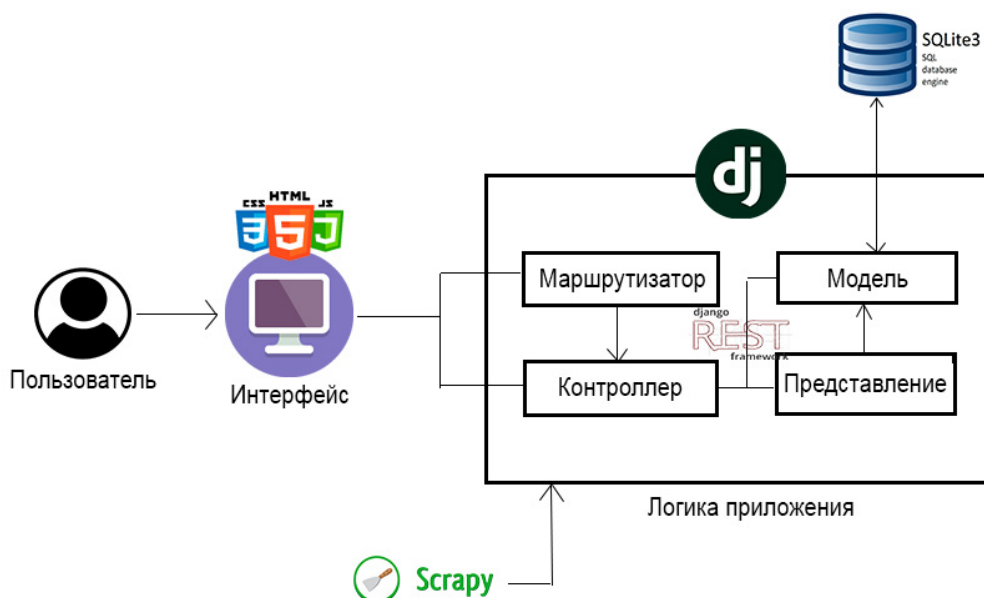


Рис. 1.2 - Архитектура веб-приложения

Как видно по схеме, согласно рис. 1.2, первым компонентом архитектуры является пользователь, затем идет Frontend, являющийся графическим интерфейсом, иначе – веб-сервером. Backend – это следующий этап в цепочке, который отвечает за работу внутренней логики приложения, и, по факту,

является внутренним сервером. Все виды запросов, кроме одного, указанного ранее, обрабатываются именно на внутреннем сервере. К таким запросам относятся операции с динамическими шаблонами страниц, работа с API и другие. Такие процессы имеют свое название - проксирование.

В задачи Frontend сервера входит:

- Возврат статических файлов;
- Проксирование – одна из самых важных частей указанной архитектуры. Сервера с поддержкой такого функционала существенно снижают нагрузку на внутренний сервер и позволяют параллельно обрабатывать часть информации;
- Распределение загрузки. Клиентский сервер ищет менее загруженные внутренние сервера и равномерно распределяет нагрузку между ними;
- Кэширование;

1.6 Базы данных

Еще в прошлом столетии к людям из соответствующей научной сферы пришла идея решить проблему хранения большого количества данных и его обработки. Это и стало толчком к появлению экспертных систем

1.6.1 Основные понятия

База данных – это структурированный метод организации хранения и обработки данных определенной предметной области, где взаимодействующие части и их суть отделена от действий с ними. Характеристиками базы данных являются ее методологи, технологическая часть и практика.

Основным различием между БД и систем, основанных на файлах, является в количестве представлений о хранящейся информации: в системе на основе файлов – несколько, в БД – одно. Данная методология реализуется с помощью специальных инструментов и технологий доступа к данным.

Система, которая предоставляет доступ к БД и осуществляет обработку и манипуляции с данными, называется СУБД – системой управления базой данных.

Для таких систем есть несколько требований, стоит выделить некоторые из них, такие как:

- Ориентированность и функциональность в соответствующей предметной области;
- Минимальная избыточность информации;
- Безопасность;
- Наличие интерфейса;
- Последовательность отдаваемой информации;
- Возможность изменения данных;

Базовой составляющей каждой БД являются модели данных, которые определяют структуру представляемой информации и варианты ее применения. Если давать ей определение, то можно сказать, что это совокупность способов и инструментов для логической интерпретации существующей информации по соответствующей предметной области.

Модель данных содержит три составляющих:

- Условия распределения информации по категориям;
- Набор существующих операций, которые можно задействовать в работе с БД;
- Объекты, определяющие набор вероятных состояний базы данных;

Базу данных необходимо как-то определять. Делается это с помощью специального языка описания данных. Это описание называется схемой БД. Она проводит связь между типологией используемой информации и особенностями свойств самой БД.

Схемы базы данных подразделяются на виды: логическая, концептуальная и физическая.

В начале работы над базой данных выделяются требования по ее концепции. На основе этих условий строится концептуальная модель, которая на уровне абстракции позволяет увидеть, как данные, связанные между собой, будут выглядеть.

Затем, на основе полученного на прошлом шаге представления, строится логическая модель, которая привязывается к техническим возможностям конкретной системы управления.

Последним шагом является определение размещения хранящихся данных и способы взаимодействия с ними. На этом шаге строится финальная - физическая модель.

Стоит ввести понятие сущности. Сущность – это основной элемент на взаимодействии которых между собой строится концептуальная модель. Это взаимодействие называется связью. Есть несколько типов связей: (1:1), (1:M), (N:M). Первая называется «один к одному», в этом случае каждому из объектов сущности представлен один объект другой сущности. Вторая имеет название «один ко многим», в ней, соответственно, каждому объекту сущности представлено количество объектов других сущностей, отличное от одного. Третья связь называется связью «многие ко многим», она предполагает соответствие нескольких объектов одного объекта нескольким объектам другого.

Можно выделить несколько принципов реляционных БД-моделей:

1. Информация, представляемая на физическом и логическом уровнях самостоятельны между собой;
2. Модель должна иметь понятную;
3. Действия над хранящимися данными реализуются и описываются с помощью инструментов высокоуровневых ЯП;

1.6.2 Модели данных

Можно сказать, что реляционная БД – это набор табличных объектов, которые взаимодействуют между собой и включают в себя информацию о сущностях одного типа. В таком табличном представлении информацию и свойства определенного объекта хранит строка, а свойства совокупности объектов, в свою очередь, хранит столбец. Каждый табличный объект содержит в своей структуре первичный ключ, который тождественно определяет каждый из объектов в структуре.

Повторения, двойственность данных и неоптимизированные трудозатраты должны отсутствовать в БД-таблицах, иначе связи между объектами таблицы будут выстроены неверно, либо ненормализованное.

Существует три нормальные формы отношений: первая (Заполненные ключевые поля и однозначность заполнения каждого из полей строк таблицы.), вторая (все поля таблицы, не включая поле первичного ключа, соединены с первичным ключом абсолютным функциональным соотношением. Также наличие первой нормальной формы.), третья (второстепенные поля не связаны друг с другом функционально. Также наличие второй нормальной формы).

2 ПРАКТИЧЕСКАЯ ЧАСТЬ

2.1 Проектирование архитектуры

Была поставлена задача разработать веб-сервис построения плана подготовки к собеседованию на основе описания вакансии. Для этого необходимо выполнить следующие шаги:

- Организовать сбор, обработку, хранение и анализ данных
- Спроектировать архитектуру веб-приложения.
- Разработать графический интерфейс, являющийся способом взаимодействия пользователя с сервисом.

- Исследовать существующие методы анализа текста и распознавание требований в описании вакансии

Каждый шаг будет рассматриваться по отдельности.

2.1.1 Архитектура приложения

Поэтапно составим схему архитектуры приложения. Имеется пользователь, которому нужно как-то взаимодействовать с приложением. Эту функцию осуществляет графический интерфейс, иначе Frontend. Средством реализации данной части приложения были выбраны следующие инструменты: JavaScript фреймворк React, язык разметки HTML и файлы стилей CSS с библиотекой Bootstrap. Эти инструменты были выбраны ввиду своей простоты и многофункциональности. Они являются стандартным набором для разработки графических интерфейсов, который на выходе дает хороший результат.

Интерфейс помимо взаимодействия с пользователем выполняет и функцию связки в внутренней программной частью приложения. Для реализации внутренней логики было решено использовать язык Python и MVC фреймворк Django. Причин выбора именно этих средств несколько. Django позволяет создавать веб-приложение, используя только внутренний функционал, не подключая сторонние библиотеки. С помощью него можно задать структуру всего проекта. Клиент (например сайт) посылает HTTP-запросы приложению. Оно же на основе этих запросов выполняет действия, определенные URL-адресом или данными в Post или Get запросах. Затем возвращая в ответе обрисованный по тому же URL'у HTML шаблон, в который подставляются данные, полученные на предыдущем шаге. При использовании Django для создания таких веб-приложений, все эти этапы обрабатываются кодом, находящимся в отдельных файлах, называемых приложениями. Python же является одним из самых простых в использовании,

а также содержит огромное количество встроенных библиотек, которые делают его функционал практически неограниченным.

Существует большое разнообразие различных инструментов для работы с базами данных. Можно выделить следующие достаточно популярные решения MySQL, Oracle, Microsoft SQL Server, PostgreSQL. Для реализации проекта было выбран инструмент SQ Lite, поскольку он имеет удобные средства для масштабирования, существенной оптимизации работы, реляционно-объектную структуру. SQ Lite, со своим богатым набором функций, может предоставить более чем достаточный функционал, при этом будучи достаточно простой для работы с одним файлом и связанной библиотекой.

2.1.2 Схема базы данных

Для разработки схемы базы данных следует определить, какую информацию мы планируем хранить в приложении. Был выделен следующий набор сущностей:

Пользователи, представленные на рис. 2.1. Для каждого пользователя будет храниться его пароль, последний вход в систему, является ли он суперюзером, ФИ, является ли он активным пользователем, дата регистрации и адрес электронной почты. Модель переопределена из стандартного класса AbstractUser:

Имя	Тип данных	Первичный ключ
id	integer	
password	varchar (128)	
last_login	datetime	
is_superuser	bool	
first_name	varchar (150)	
last_name	varchar (150)	
is_staff	bool	
is_active	bool	
date_joined	datetime	
email	varchar (254)	

Рис. 2.1 - Структура таблицы пользователей

Вакансии, представленные на рис. 2.2. Для каждой вакансии будут храниться ее описание, скиллы, ссылки на вакансию и названия.

Имя	Тип данных	Первичный ключ
id	integer	
description	varchar (3000)	
skills	varchar (60)	
url	varchar (60)	
name	varchar (50)	

Рис. 2.2 - Структура таблицы вакансий

Для занесения данных в БД изначально было решено использовать автоматический внос данных в модель Vacancy loaddata через fixtures. Файлы предварительной настройки (fixtures) содержат набор данных, которые Django может импортировать в базу данных. Только этот вариант работает со специфическим представлением данных в json, то есть возникла необходимость в сериализации данных, полученных на предыдущем шаге. Был написан скрипт, который трансформировал данные в формате JSON в данные, которые могут быть прочитаны моделью.

Представление данных до парсинга:

```
[
{"name": "Frontend-разработчик (Senior)", "skills": "JavaScript", "other": "",
"url":"https://career.habr.com/vacancies/1000068662"},
{"name": "Frontend developer (Vue.js)", "skills": "JavaScript", "other": "", "url":
"https://career.habr.com/vacancies/1000069398"},
{"name": "Frontend developer (Vue.js)", "skills": "Git", "other": "", "url":
"https://career.habr.com/vacancies/1000069398"}
]
```

Представление данных после парсинга:

```
[
  {
    "model": "vacancy.Vacancy",
    "pk": 1,
    "fields": {
      "name": "Программист прикладного ПО",
      "skills": "C++",
      "url": "https://ryazan.hh.ru/vacancy/41630586",
      "description": ""
    }
  }
]
```

Возникла проблема с представлением модели в тексте файла, но после исправления ошибки представление пришло в нормальную форму. Также при переносе данных было замечено, что переносилось только одно поле - имя, после чего к коду модели были добавлены условия `null=True` (пустое значение сохраняется как `NULL` в базе данных), `blank=True` (подгружаемое поле может быть пустым). Проведя все необходимые миграции и запустив скрипт был получен файл, который без проблем вносится в базу данных.

2.2 Разработка практических решений

После построения архитектуры приложения и выбора программных средств для ее реализации можно, непосредственно, приступить к разработке графического и внутреннего программного обеспечения.

2.2.1 Этап сбора данных

Для того, чтобы заполнить базу данные веб-приложения и обучить классификатор необходимы данные. В моем случае в качестве данных было решено брать описание вакансий с сайтов поиска работы. Инструментом для

данной задачи был выбран пакет Scrapy как самый простой и оптимальный вариант.

Scrapy — это быстрый веб-фреймворк с открытым исходным кодом, написанный на Python, который используется для извлечения данных с веб-страницы с помощью селекторов на основе XPath. В Scrapy имеется встроенный механизм под названием Selector, который отвечает за поиск и извлечение данные со страниц с помощью XPath и CSS. Scrapy позволяет извлекать информацию со страниц будь они доступны или нет. Также одним из плюсов является меньшее потребление памяти и мощности процессора в сравнении с другими библиотеками. Scrapy может экспортировать данные страниц в следующие форматы: CSV, XML, JSON и JSON Line.

После установки пакета необходимо было написать скрапер, который обходит все страницы сайтов hh.ru, career.habr.com и собирает из них данные (название, скилл, ссылка, описание) для каждой вакансии. Парсер запускается командой `scrapy crawl` и записывает данные в указанный json и csv файл.

Задача была разбита на 2 части. Сначала был написан скрапер для парсинга одной страницы, затем с помощью цикла и дополнительных xpath путей был создан скрапер для всех вакансий и всех страниц.

Xpath — это язык запросов к объектам соответствующего документа. Чтобы получить интересующие данные, необходимо всего лишь создать запрос, описывающий эти данные. Пример xpath запроса для поля name:

```
(item['name'] = content.xpath('..div[contains(@class, "vacancy-
title")]h1//text()).get())
```

Возникли проблемы с тем, что описание во всех вакансиях не упорядочено, поэтому нельзя выделить отдельные категории, такие как «требования» и «условия». Поэтому описание обрабатывается полностью и нуждается в чистке.

Парсер был модифицирован для других сайтов путем замены xpath путей.

Тренировочную вакансию я беру с сайта rabota.ru. Для сайта, с которого мы берем тренировочную вакансию был взят первоначальный вариант парсера для одной ссылки.

Далее в процессе написания классификатора парсер был модифицирован следующим образом – считывались только данные вакансий, содержащие в поле skills определенные значения.

2.2.2 Создание классификатора

В машинном обучении задача классификации решается с использованием обучения с учителем, поскольку классы определяются заранее и для примеров обучающего множества метки классов заданы. Аналитические модели, решающие задачу классификации, называются классификаторами. Каждый элемент выборки может принадлежат к одному или нескольким классам. В зависимости от этого различают задачу многоклассовой и одноклассовой классификации. В моей задаче необходимо решить задачу классификации текста по признакам скиллов. Было решено использовать наивный байесовский классификатор.

Первым этапом была подготовка данных, полученных при скраппинге сайтов. В ходе этой работы было выяснено, что данные не подходят для обучения по ряду причин:

- Большое количество нулевых значений
- Классы были представлены списком
- Количество классов было очень большим, по сравнению с обучаемыми данными

Вследствие этого возникало большое число ошибок и точность вычисления в последующем не превышала сотых долей от единицы.

Задача была решена после доработки скраппера, который готовил данные

Теперь:

- Нулевых значений нет
- Классы представлены по одному на строку
- Количество классов уменьшено до 30 в данных на 3000 строк

Следующим этапом было непосредственно построение классификатора. Данные были приведены к сравниваемому типу, поделены на тестовую и тренировочную выборки. Определены классы, которые используются в предсказаниях. Для обучения, как ранее говорилось был использован байесовский классификатор в библиотеке `sklearn`.

Также были испробованы и другие алгоритмы классификации, такие как KNN, SVM, Decision Tree и т.д. Наиболее высокую точность показал только алгоритм SVM. Вследствие работы над ошибками была обнаружена ошибка в представлении классов описания и названия, было решено реализовать следующие методы приведения текста:

- Tf idf
- Word to vec
- Lda

2.2.3 Frontend

Для реализации графического интерфейса приложения были написаны стили в формате CSS с использованием библиотеки Bootstrap. Шаблон страницы был спроектирован в HTML-шаблоне. HTML-страница содержит в себе описание всех стилей, привязанных к селектору. Селекторы в свою очередь содержат специальное выражение, описывающее класса, номера, теги элементов и другие их атрибуты.

В своем приложении я создала шаблоны 6 вкладок, краткий обзор функциональности:

1. Шапка (index.html, рис. 2.3) – шапка сайта, включаемая через extends во все шаблоны сайта. Отображается всегда, интерактивность – переход по вкладкам других шаблонов.



Рис. 2.3 - Элемент интерфейса – шапка сайта

2. Домашняя страница (home.html, рис. 2.4) – страница, открываемая по умолчанию и доступная через лого сайта на хидере. Интерактивность – основная страница поиска (тема вкр), переход по вкладкам других шаблонов.

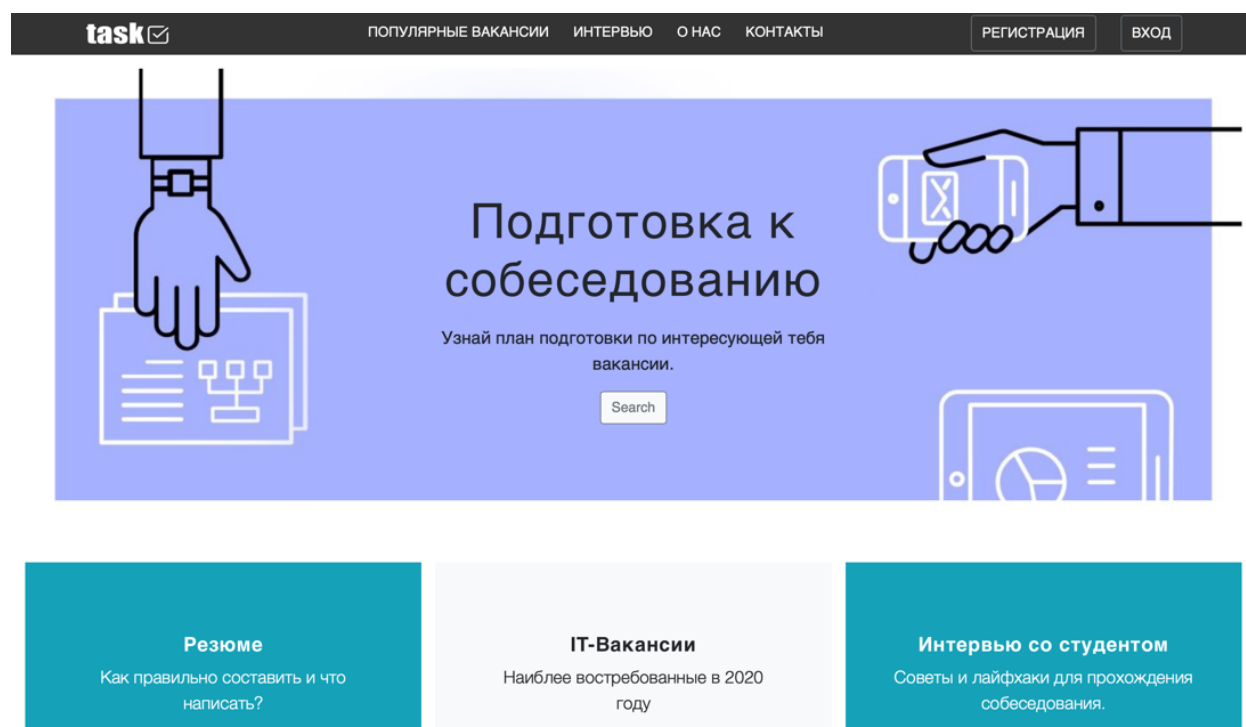


Рис. 2.4 – Интерфейс домашней страницы

3. О нас (about.html, рис. 2.5) – страница с краткой информацией о теме диплома. Доступна только из шапки сайта. Интерактивность – нет. Создана для демонстрации работы bootstrap'a

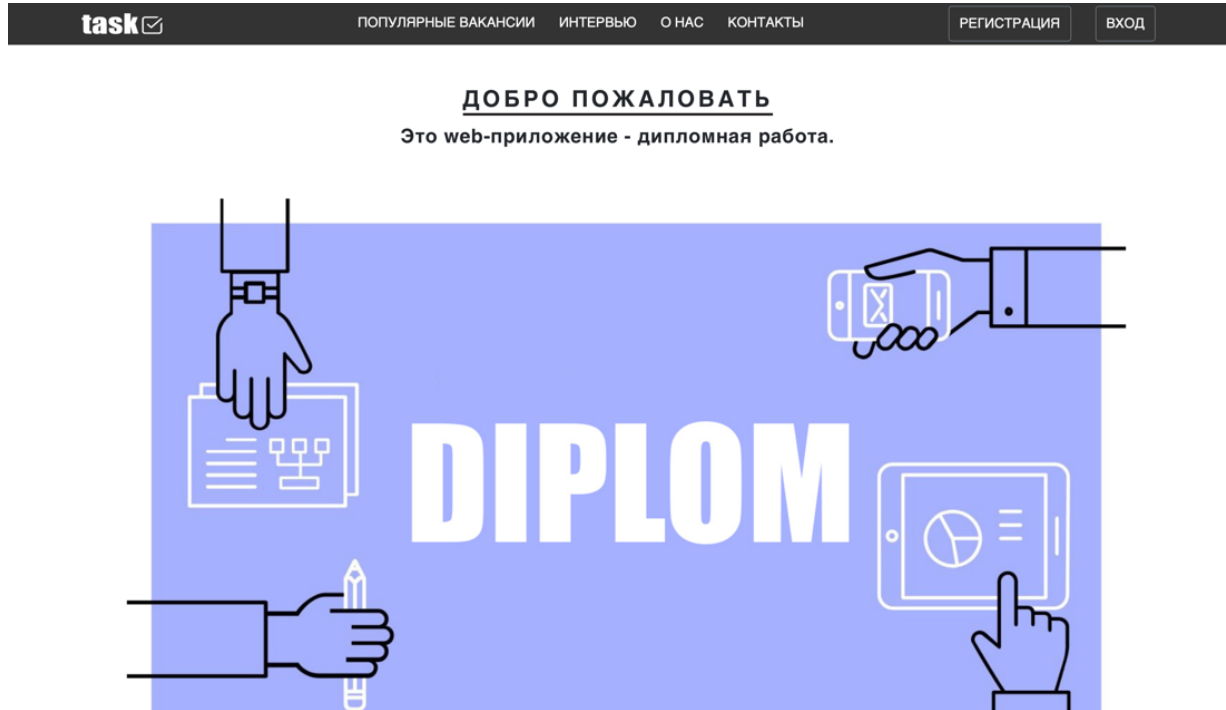


Рис. 2.5 – Интерфейс страницы «О нас»

4. Контакты (contacts.html, рис. 2.6) – страница с моими контактами и контактами ВУЗа. Доступна только из шапки сайта, интерактивность – вставлен frame с местоположением на картах google, подключенный через открытое API.

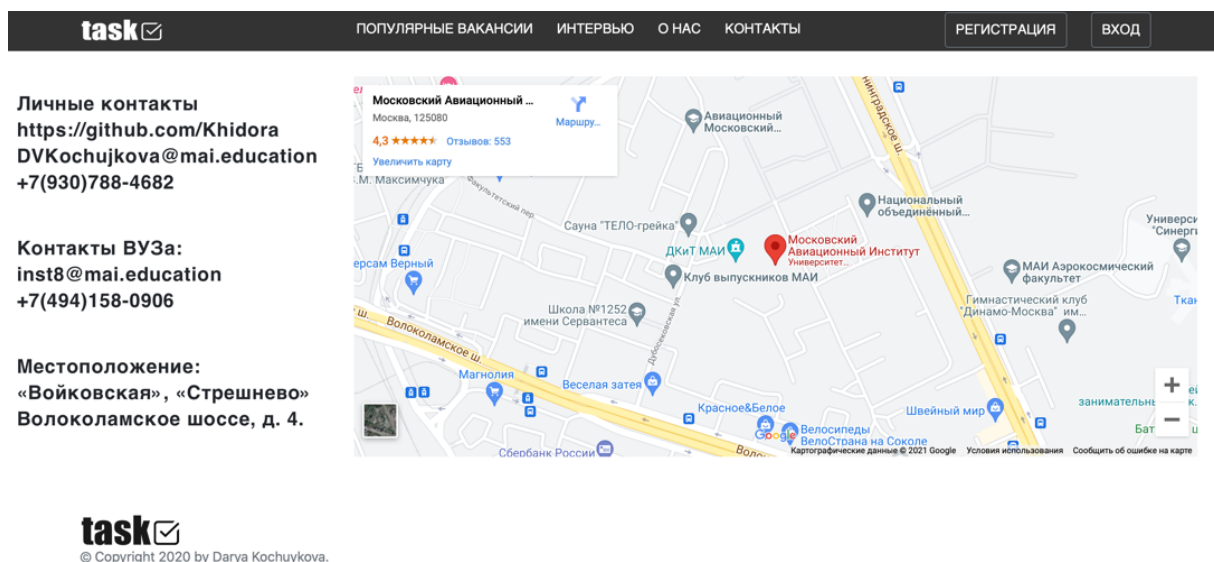


Рис. 2.6 – Интерфейс страницы «Контакты»

5. Резюме (resume.html, рис. 2.7) – страница с информацией о составлении резюме, добавлена для красивого внешнего вида приложения.

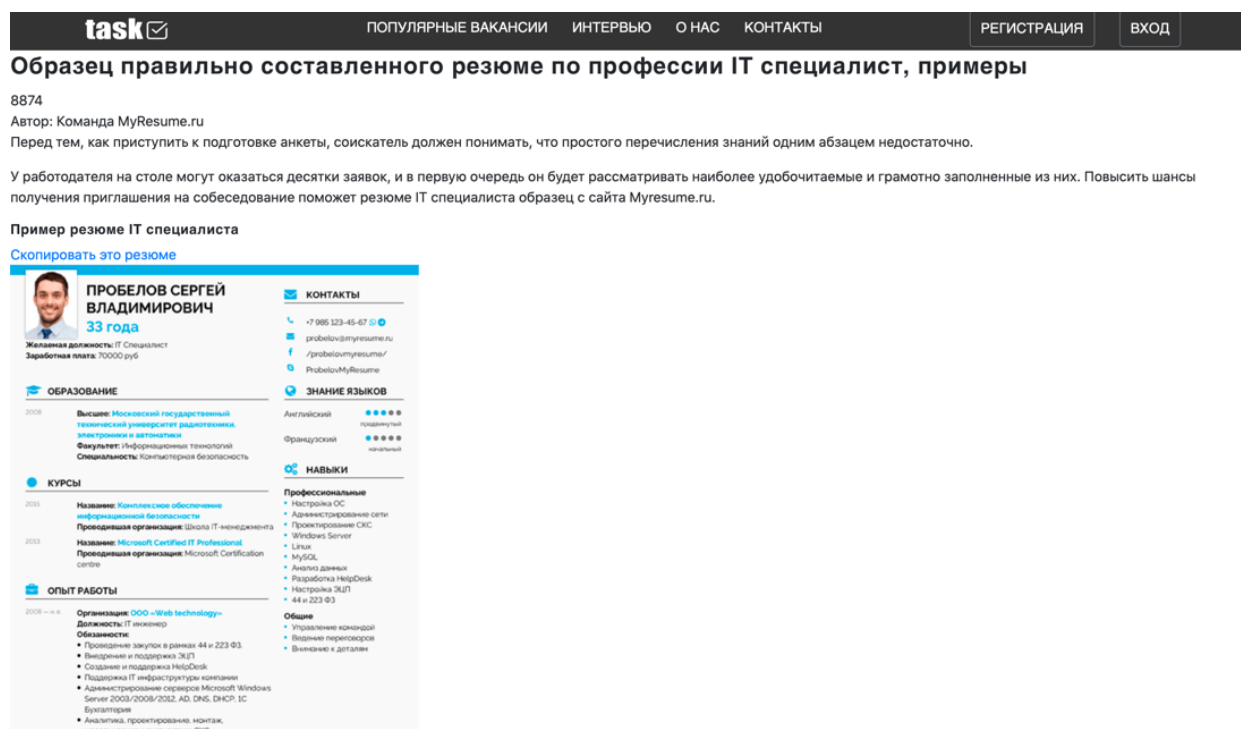


Рис. 2.7 – Интерфейс страницы «Резюме»

6. Популярные вакансии (vacan.html, рис. 2.8) – страница с популярными вакансиями. Интерактивность – вакансии выводятся с помощью java скрипта из базы данных.

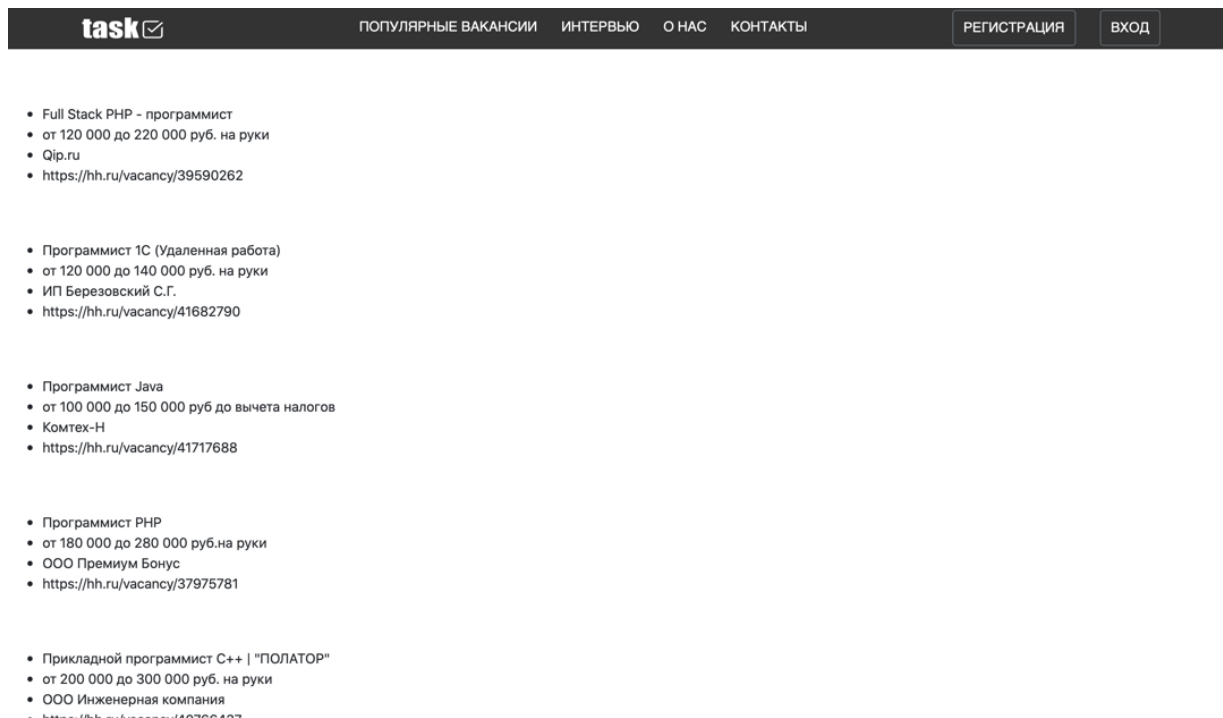


Рис. 2.8 – Интерфейс страницы «Популярные вакансии»

2.2.4 Backend

Используя фреймворк Django, в программном коде описаны компоненты, из которых будет состоять внутренняя логика приложения, а именно: маршрутизатор, контроллеры и модели. Такой набор компонентов соответствует шаблону MVC.

Всего в моем проекте реализовано 7 приложений:

1. Diplom – приложение с общими настройками проекта и маршрутизатором.
2. Nabrcareer, parserscrapy, rabotaru – приложения, в которых реализован сбор данных.
3. Pages – приложение, где лежат все шаблоны frontend'а, и файлы стилей, а также статические файлы.

4. Vacansy – приложение, в котором проводится работа над базой данных, все скрипты для преобразования данных и модели данных находятся там.
5. User – приложение, где переопределен класс AbstractUser

В соответствии с описанным API был реализован маршрутизатор с url'ами из других приложений. Маршрутизаторы составляют иерархию для оптимизированного поиска совпадений и разделения логики приложения. Также, соответственно, для каждого приложения и шаблона есть свои url'ы, прописанные в маршрутизаторах нижнего уровня, где к каждому запросу сопоставлен контроллер.

Также у приложения есть admin и настроенная для него api. В нем также можно редактировать модели данных, применять миграции и заполнять базу данными.

В соответствии с описанным API был реализован маршрутизатор (Router), представленный ниже:

```
urlpatterns = [
    path('admin/', admin.site.urls),
    path("", include('pages.urls')),
    path('about/', include('pages.urls')),
    path('contacts/', include('pages.urls')),
    path('interview/', include('pages.urls')),
    path('resume/', include('pages.urls')),
    path('vacan/', include('pages.urls')),
]
```

Маршрутизаторы могут составлять иерархию для более оптимизированного поиска совпадений и разделения логики приложения. Код маршрутизатора нижнего уровня выглядит следующим образом:

```
urlpatterns = [
    path("", views.first_view),
    path('about/', views.second_view),
```

```

    path('contacts/', views.third_view),
    path('interview/', views.fourth_view),
    path('resume/', views.five_view),
    path('vacan/', views.six_view)
]

urlpatterns = [
    path('vacancies/', views.VacView.as_view(), name = 'view'),
]

```

Были созданы следующие модели для взаимодействия с базой данных:

Модель «Вакансия»:

```

class Vacancy(models.Model):
    name = models.CharField(max_length=50, null=True, blank=True)
    skills = models.CharField(max_length=60, null=True, blank=True)
    url = models.CharField(max_length=60, null=True, blank=True)
    description = models.CharField(max_length=3000, null=True, blank=True)

```

Модель «Скилл»:

```

class Skill(models.Model):
    skill = models.CharField(max_length=40)
    #vac = models.ManyToManyField(Vacancy)

    def __str__(self):
        return self.skill

```

Модель «Сайт»:

```

class Site(models.Model):
    url = models.CharField(max_length=70)

```

```
def __str__(self):
    return self.url
```

```
class CustomUser(AbstractUser):
    username = None
    email = models.EmailField(_('email address'), unique=True)
    USERNAME_FIELD = 'email'
    REQUIRED_FIELDS = []
    objects = CustomUserManager()
    def __str__(self):
        return self.email
```

Таким образом мы получаем готовую программно-аппаратную часть, написанную на Python с использованием MVC фреймворка Django.

Удобен факт того, что нам не требуется непосредственно работать с базой данных и писать SQL запросы. Этим занимается ORM модуль Django.

2.2.5 Результат работы

В ходе работы был выбран следующий способ выдачи рекомендаций: есть список URL-адресов вакансий, по нажатию на которые мы обращаемся к тому же адресу в базе данных. По адресу определяем название и навыки вакансии, после этого, обращаясь к таблице скиллов выбираем тройку случайных объектов, составляющих структуру ответа: вопроса, задачи и рекомендуемой литературы. После этого данные поля выводятся в соответствующих местах прописанного ранее HTML-шаблона, представленного на рис. 2.9

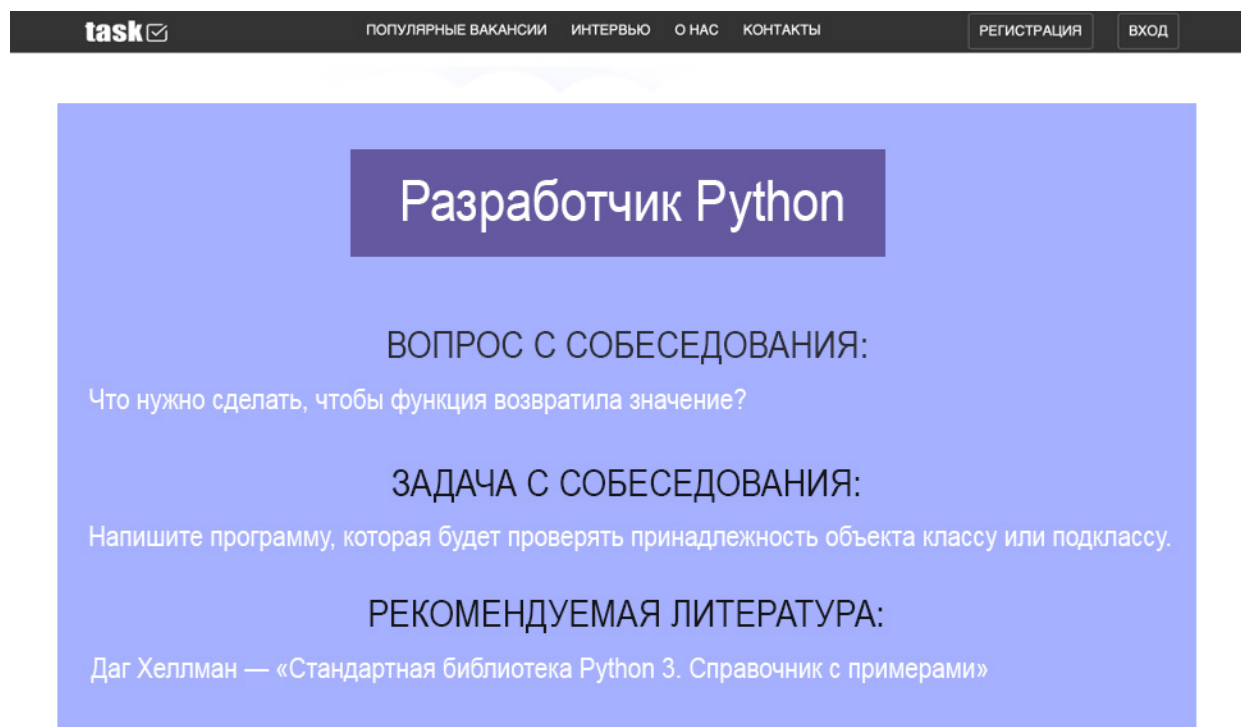


Рис. 2.9 – Результат работы

ЗАКЛЮЧЕНИЕ

В результате проделанной работы были выполнены поставленные задачи:

1. Спроектирован программный интерфейс приложения связывающий программный интерфейс (клиентскую часть) и программно-аппаратную часть приложения.
2. Реализована программно-аппаратная часть сервиса, отвечающая за операции с данными и вычислительные механизмы.
3. Разработан программный интерфейс приложения, служащий связующим звеном между приложением и конечным пользователем.
4. Реализованы сбор, обработка и хранение данных.
5. Реализован алгоритм анализа полученных данных.

Представленный программный комплекс позволяет пользователю построить план подготовки к собеседованию на основе описания вакансий со стороннего сайта.

В будущем планируется улучшать приложение добавлением новых функциональных возможностей.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Морвиль П., Розенфельд Л. Информационная архитектура в Интернете, 3-е издание. – Пер. с англ. – СПб:Символ-Плюс, 2010. – 608 с.
2. Мартин Клеппман. Высоконагруженные приложения. Программирование, масштабирование, поддержка. - СПб: Питер, 2019. – 640 с.
3. Форд Нил, Парсонс Ребекка, Куа Патрик. Эволюционная архитектура. Поддержка непрерывных изменений. — СПб.: Питер, 2019. — 272 с.
4. Митчелл Р. Скрайпинг веб-сайтов с помощью Python. – М.: ДМК Пресс, 2016. – 280 с.
5. Heather Adkins, Betsy Beyer, Paul Blankinship, Piotr Lewandowski, Ana Oprea, and Adam Stubblefield. Building Secure and Reliable Systems. 2020
6. Ньюмен С. Создание микросервисов. — СПб.: Питер, 2016. — 304 с.
7. Марманис Х., Бабенко Д. Алгоритмы интеллектуального Интернета. Передовые методики сбора, анализа и обработки данных. – Пер. с англ. – СПб.: Символ-Плюс, 2011. – 480 с.

ПРИЛОЖЕНИЯ

ПРИЛОЖЕНИЕ 1

Программный код скраппера

```
import scrapy
from scrapy.item import Field

class JobItem(scrapy.Item):
    name = Field()
    skills = Field()
    url = Field()
    description = Field()

class ItvacancySpider(scrapy.Spider):
    name = 'itvacancy'
    allowed_domains = ['hh.ru']
    start_urls = [

    def parse(self, response):
        for href in response.xpath('//a[@data-qa="vacancy-serp__vacancy-
title"]/@href'):
            url = response.urljoin(href.extract().split('?')[0])
            yield scrapy.Request(url, callback=self.parse_item)

        next_page = response.xpath('//a[@data-qa="pager-next"]/@href').get()
        if next_page is not None:
            yield response.follow(next_page, callback=self.parse)

    def parse_item(self, response):
        i = 1

        if response.xpath('//*[contains(@data-qa, "skills-element")]/span/text()').get()
```

!= None:

```
for i in range(len(response.xpath('//*[@id="HH-React-Root"]//div/div/div/div/div[1]/div[3]/div[2]/div/div/div[4]/div/div/div/div/span/text(
))))+1):
```

```
    item = JobItem()
```

```
    content = response.xpath('(//div[@class="main-content"]//div[contains(@class, "bloko-column_container")])[1]')
```

```
    vacancy_section = content.xpath('(//div[@class="vacancy-description"]//div[@class="vacancy-section"])[1]/div[1]')
```

```
    item['name'] = content.xpath('(//div[contains(@class, "vacancy-title")]//h1/text()).get()
```

```
    item['skills'] = content.xpath('//*[@id="HH-React-Root"]//div/div/div/div/div[1]/div[3]/div[2]/div/div/div[4]/div/div/div/div['+str(i)+']/span/text()).get()
```

```
    item['url'] = response.request.url
```

```
    item['description'] = vacancy_section.get()
```

```
    listv = ['C++', 'C', 'C#', '1C', '3D', 'API', 'Bash', 'C++11', 'CSS', 'CUDA', 'Delphi', 'Debian', 'Django', 'Figma', 'Flask', 'GCC', 'GO', 'GUI', 'Git', 'HTML', 'Java', 'Kafka', 'Keras', 'Kotlin', 'Linux', 'MATLAB', 'ML', 'MySQL', 'Nginx', 'NodeJs', 'ORM', 'OpenGL', 'Pascal', 'Perl', 'Phyton', 'REST', 'React', 'Ruby', 'SQL', 'Shell', 'Swift', 'TCP/IP', 'UI', 'Unity', 'VueJS', 'Web', 'JavaScript', '1C-Битрикс', 'Английский язык', 'ООП', 'jQuery']
```

```
    if (item['skills'] != None) and (item['skills'] in listv):
```

```
        yield item
```